Aether — Quick Start PoC

From Zero to Deployed in 5 Minutes

Install, deploy, migrate, and monitor — a complete proof of concept in minutes.

Universal Runtime Control Plane

Three Ways to Install

Choose the method that fits your environment. All options give you the full Aether CLI.

📦

Option A: Container Image

Pull the official image and alias the CLI — no local dependencies required.

```
docker pull ghcr.io/aether/aether:latest

# Create a convenient alias
alias aether='docker run --rm -v $(pwd):/work \
  -v /var/run/podman/podman.sock:/var/run/podman/podman.sock \
  ghcr.io/aether/aether:latest'

aether --version
# aether 0.9.0 (runtime: container)
```

📥

Option B: Pre-built Binary

Download a statically-linked binary for Linux x86_64 or aarch64.

```
# Download latest release
curl -fsSL https://get.aether.dev | sh

# Or fetch a specific version
curl -LO https://github.com/aether/aether/\
releases/download/v0.9.0/aether-linux-amd64

chmod +x aether-linux-amd64
sudo mv aether-linux-amd64 /usr/local/bin/aether

aether --version
# aether 0.9.0 (runtime: binary)
```

🔧

Option C: Build from Source

Clone the repo and compile with Cargo. Requires Rust 1.75+.

```
# Clone the repository
git clone https://github.com/aether/aether.git
cd aether

# Build in release mode
cargo build --release

# Install to PATH
sudo cp target/release/aether /usr/local/bin/

aether --version
# aether 0.9.0 (runtime: source)
```

> *All three methods produce the same CLI binary with identical capabilities. Choose container for isolation, binary for speed, or source for customization.*

Your First Deployment

Define a workload in YAML, validate, build, run, and inspect — all from one tool.

1. Create workload.yaml

```yaml
apiVersion: aether/v1
kind: Workload
metadata:
  name: my-web-app
  labels:
    team: platform
    env: staging
spec:
  runtime: podman
  image: docker.io/library/nginx:alpine
  replicas: 2
  ports:
    - containerPort: 80
      hostPort: 8080
  resources:
    cpu: "500m"
    memory: "256Mi"
  healthCheck:
    httpGet:
      path: /
      port: 80
    intervalSeconds: 10
    timeoutSeconds: 3
  env:
    - name: APP_ENV
      value: staging
```

2. Validate, Build, Run, Inspect

Validate the spec

```
$ aether validate workload.yaml
✓ Schema valid
✓ Image reference valid
✓ Port range OK
✓ Resource limits OK
Result: PASS (4/4 checks)
```

Build and Run

```
$ aether build workload.yaml
Building my-web-app... done (1.2s)

$ aether run workload.yaml
Deploying my-web-app to podman...
  ✓ Container created (replica 1/2)
  ✓ Container created (replica 2/2)
  ✓ Health check passing
Status: RUNNING
```

Check status

```
$ aether status my-web-app
NAME         RUNTIME   REPLICAS   STATUS    AGE
```

Stream logs

```
$ aether logs my-web-app --follow
[replica-1] 172.17.0.1 - GET / 200 0.002s
```

```
my-web-app   podman   2/2       Running  45s

Endpoints:
  http://localhost:8080 → :80
```

```
[replica-2] 172.17.0.1 - GET / 200 0.001s
[replica-1] health-check OK (12ms)
[replica-2] health-check OK (9ms)
```

Live Migration Demo

Move workloads between runtimes with a single command. Zero downtime, full state preservation.

Migrate from Podman to Kubernetes

```
$ aether migrate my-web-app kubernetes --strategy blue-green

Migration Plan:
  Source:      podman (local)
  Target:      kubernetes (cluster: staging-east)
  Strategy:    blue-green
  Rollback:    automatic (on failure)

Phase 1/4: Preparing target environment...
  ✓ Namespace created: aether-my-web-app
  ✓ Deployment manifest generated
  ✓ Service + Ingress configured

Phase 2/4: Deploying green environment...
  ✓ Pod my-web-app-green-7f8d9 running (1/2)
  ✓ Pod my-web-app-green-a3b12 running (2/2)
  ✓ Health checks passing

Phase 3/4: Switching traffic...
  ✓ Traffic routed to green (100%)
  ✓ Blue environment draining

Phase 4/4: Cleanup...
  ✓ Podman containers stopped
  ✓ Migration state recorded

Result: SUCCESS (total: 38s)
```

Verify Migration

```
$ aether status my-web-app
NAME         RUNTIME      REPLICAS  STATUS   AGE    MIGRATED-FROM
my-web-app   kubernetes   2/2       Running  12s    podman (38s ago)
```

> *Aether supports migrations between any pair of runtimes. Strategies include rolling, blue-green, canary, and immediate. Automatic rollback protects against failures.*

Dashboard & API

Monitor workloads with the built-in TUI or integrate via the REST API.

Terminal UI (TUI)

Full-featured dashboard in your terminal. Real-time metrics, logs, and management.

```
$ aether tui

┌─ Workloads ──────────────────────────┐
│ my-web-app  k8s    2/2  Running      │
│ api-svc     podman 1/1  Running      │
│ cache       podman 3/3  Running      │
└──────────────────────────────────────┘
  [Tab] Switch  [Enter] Detail
  [l] Logs  [r] Restart  [q] Quit
```

Web Dashboard

Launch the web UI on port 5090 for browser-based monitoring.

```
$ aether serve --port 5090

Aether Web Dashboard
  URL:  http://localhost:5090
  API:  http://localhost:5090/api/v1
  Auth: Bearer token (see below)

Generating API token...
  Token: aether_tk_a8f3...b92e
  Save this token — it will not be
  shown again.
```

REST API Examples

List all workloads

```
$ curl -s -H "Authorization: Bearer $TOKEN" \
  http://localhost:5090/api/v1/workloads | jq

{
  "workloads": [
    {
      "name": "my-web-app",
      "runtime": "kubernetes",
      "replicas": { "ready": 2, "desired": 2 },
      "status": "Running",
      "age": "5m12s"
    }
  ],
  "total": 1
}
```

Deploy via API

```
$ curl -X POST \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/yaml" \
  --data-binary @workload.yaml \
  http://localhost:5090/api/v1/workloads

{
  "status": "created",
  "name": "my-web-app",
  "runtime": "podman",
  "message": "Deployment initiated"
}
```

5090	REST	JWT	OpenAPI
Default Port	API Protocol	Auth Method	Schema Available

What's Next?

You have a running workload. Now unlock the full power of Aether.

Explore Templates

8 built-in templates for common workload patterns: web apps, APIs, databases, workers, cron jobs, and more.

```
$ aether template list
$ aether template web-app --name my-svc
```

Multi-Workload Compose

Define entire stacks in aether-compose.yaml with dependency ordering and shared networking.

```
$ aether compose up
$ aether compose status
```

Secrets Management

Encrypt and inject secrets at deploy time. Supports file-based, environment, and vault-backed secrets.

```
$ aether secret set DB_PASS --value s3cret
$ aether secret list
```

CI/CD Integration

Use --yes for non-interactive mode. Dry-run support for pipeline validation before deploy.

```
$ aether run workload.yaml --yes --dry-run
$ aether run workload.yaml --yes
```

Ready to Go Further?

Check out the Observability guide for metrics, the Developer Experience guide for 40+ commands, or dive into the full documentation at docs.aether.dev.